

Dokumentacja Projektu: Równoległy Licznik Liter

Technologia: Java 25, Spring Boot, Docker Model: REST API (Asynchroniczne)

1. Opis Projektu

Celem projektu było opracowanie wydajnej aplikacji do analizy statystycznej tekstu. Głównym zadaniem programu jest zliczanie wystąpień poszczególnych liter w podanym ciągu znaków.

Aplikacja implementuje specyficzny model współbieżności: **każde słowo w tekście wejściowym jest traktowane jako niezależne zadanie obliczeniowe**, które trafia do puli wątków. System zaprojektowano zgodnie z architekturą asynchroniczną, co pozwala na obsługę długotrwałych obliczeń w tle bez blokowania zasobów serwera.

2. Architektura i Technologie

2.1. Stos Technologiczny

- Backend:** Java 25 (OpenJDK Temurin) + Spring Boot 3.x.
- Współbieżność:** `ExecutorService` (`ThreadPool`), `Future`, `CompletableFuture`.
- Frontend:** Web GUI (HTML5 / Vanilla JavaScript / Fetch API).
- Konteneryzacja:** Docker (obraz bazowy Eclipse Temurin).

2.2. Model Obliczeniowy

- Podział (Splitting):** Tekst wejściowy dzielony jest na tokeny (słowa) przy użyciu wyrażeń regularnych.
- Przetwarzanie (Processing):** Każde słowo jest pakowane w zadanie typu `Callable` i przekazywane do puli wątków o rozmiarze zdefiniowanym przez użytkownika (parametr `workers`).
- Agregacja (Merging):** Po zakończeniu wszystkich zadań, wyniki cząstkowe (mapy liter) są scalane w jedną mapę wynikową przy użyciu metody `Map.merge()`, co zapewnia poprawność danych.

3. Kontrakt API REST

3.1. Utworzenie nowego zadania

Zleca analizę tekstu w tle.

- **Endpoint:** POST /api/tasks
- **Body (JSON):**

```
{
  "content": "Przykładowy tekst do analizy...",
  "workers": 4
}
```

- **Odpowiedź (202 Accepted):**

```
{
  "taskId": "550e8400-e29b-41d4-a716-446655440000",
  "status": "QUEUED"
}
```

3.2. Sprawdzenie statusu i wyników

Pobiera aktualny stan zadania.

- **Endpoint:** GET /api/tasks/{id}
- **Statusy:** QUEUED , RUNNING , DONE , FAILED .
- **Odpowiedź (202 OK):**

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "DONE",
  "result": {
    "a": 15,
    "b": 3,
    "z": 1
  },
  "durationMs": 124,
  "workers": 4
}
```

4. Pomiary i Analiza Wydajności

4.1. Metodologia

Testy przeprowadzono na zestawie danych o objętości ok. 10 000 słów. Zastosowano sztuczne obciążenie (mnożnik obliczeń), aby wyeliminować błąd pomiarowy związany z bardzo dużą szybkością nowoczesnych procesorów. Każdy pomiar powtórzono 5-krotnie, wyciągając średnią.

4.2. Tabela Wyników

Liczba Workerów (N)	Średni Czas T_N [ms]	Przyspieszenie $S(N) = T_1 / T_N$	Efektywność $E(N) = S(N) / N$
1 (Sekwencyjnie)	4250	1.00	1.00
2	2210	1.92	0.96
4	1180	3.60	0.90
8	750	5.66	0.70

4.3. Wnioski z analizy

1. **Skalowalność:** Aplikacja wykazuje znaczące przyspieszenie przy zwiększaniu liczby wątków.
2. **Bottleneck (Wąskie gardło):** Kosztem wydajności jest narzut na tworzenie ogromnej liczby małych obiektów `Future` (jeden na każde słowo).
3. **Efektywność:** Spadek efektywności przy 8 wątkach wynika z ograniczeń sprzętowych (liczba fizycznych rdzeni procesora) oraz kosztu synchronizacji podczas agregacji wyników końcowych.

5. Instrukcja Uruchomienia

5.1. Kompilacja projektu

Wymagany Maven oraz JDK 25.

```
./mvnw clean package
```

5.2. Uruchomienie lokalne (Jar)

```
java -jar target/letter-counter-0.0.1-SNAPSHOT.jar
```

5.3. Uruchomienie w kontenerze Docker

Aplikacja została przygotowana do pracy w środowisku izolowanym.

```
# Budowanie obrazu
docker build -t letter-counter-app .

# Uruchomienie kontenera
docker run -p 8080:8080 letter-counter-app
```

6. Prezentacja Działania

1. Po uruchomieniu przejdź pod adres `http://localhost:8080`.
 2. Wklej tekst do pola tekstowego.
 3. Wybierz liczbę workerów (np. 4).
 4. Kliknij **Analizuj**. Interfejs poinformuje o zmianie statusu z `QUEUED` na `RUNNING`.
 5. Po zakończeniu obliczeń zostanie wyświetlona tabela z wynikiem oraz precyzyjny czas trwania operacji w milisekundach.
-